Chapitre 2 : Les structures de contrôle

Hassen NAKBI ::: hassen.nakbi@gmail.com

1ere année Licence

2 octobre 2023

Introduction

Choix d'une instruction

Les instructions d'un algorithme sont enchaînées de manière linéaire, les unes à la suite des autres. Nous souhaitons maintenant comprendre comment sélectionner une instruction particulière parmi un ensemble ou répéter une instruction pour atteindre un résultat spécifique.

2 Structures de contrôle conditionnelle

- Définition
- Structure simple
- Structure alternative
- Structure au choix multiple

Définition

Structures conditionnelles

Les structures de contrôle conditionnelles permettent de prendre des décisions en fonction de certaines conditions. Elles permettent à un algorithme de suivre différentes voies ou branches d'exécution en fonction des valeurs des données en cours de traitement.
Il existe principalement trois types de structures conditionnelles :

1. Structure simple.
2. Structure alternative.
3. Structure au choix multiple.

2 Structures de contrôle conditionnelle

- Définition
- Structure simple
- Structure alternative
- Structure au choix multiple

Structure simple

Une structure conditionnelle simple est utilisée pour exécuter un ensemble d'instructions si une seule condition spécifiée est vraie. Si la condition n'est pas vraie, l'ensemble d'instructions n'est pas exécuté.

Structure simple

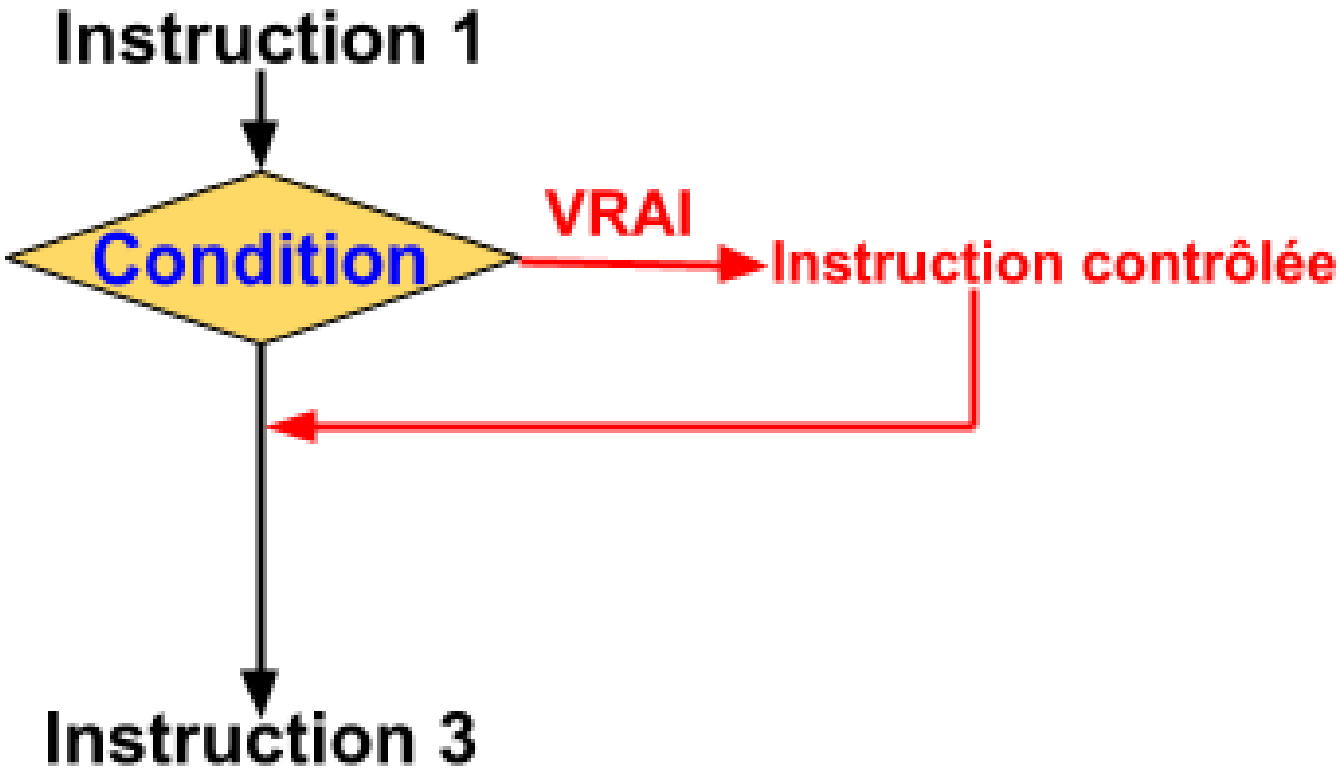


Figure – Organigramme structure simple

Structure simple

Syntaxe de la structure simple

Algorithmique

- DÉBUT
- SI (condition) alors
- instructionVraie
- FinSI
- FIN

Langage C

- int main( ) {
- if (condition) {
- instructionVraie ;}
- return 0 ;}

Structure simple

Exemple

Algorithmique

- VAR x :entier
- DÉBUT
- x $\longleftarrow$ 11
- SI (x MOD 2 = 0) alors
- Écrire(x ,"est pair")
- FinSI
- FIN

Langage C

- short x = 11 ;
- int main( ) {
- if (x % 2 == 0) {
- printf("%hi est pair",x) ;}
- return 0 ;}

2 Structures de contrôle conditionnelle

- Définition
- Structure simple
- Structure alternative
- Structure au choix multiple

Structure alternative

Définition

Une structure conditionnelle alternative permet d'exécuter un ensemble d'instructions si une condition spécifiée est vraie, et un autre ensemble d'instructions si cette condition est fausse. Cela permet au programme de suivre deux voies d'exécution distinctes en fonction de la condition.

Structure alternative

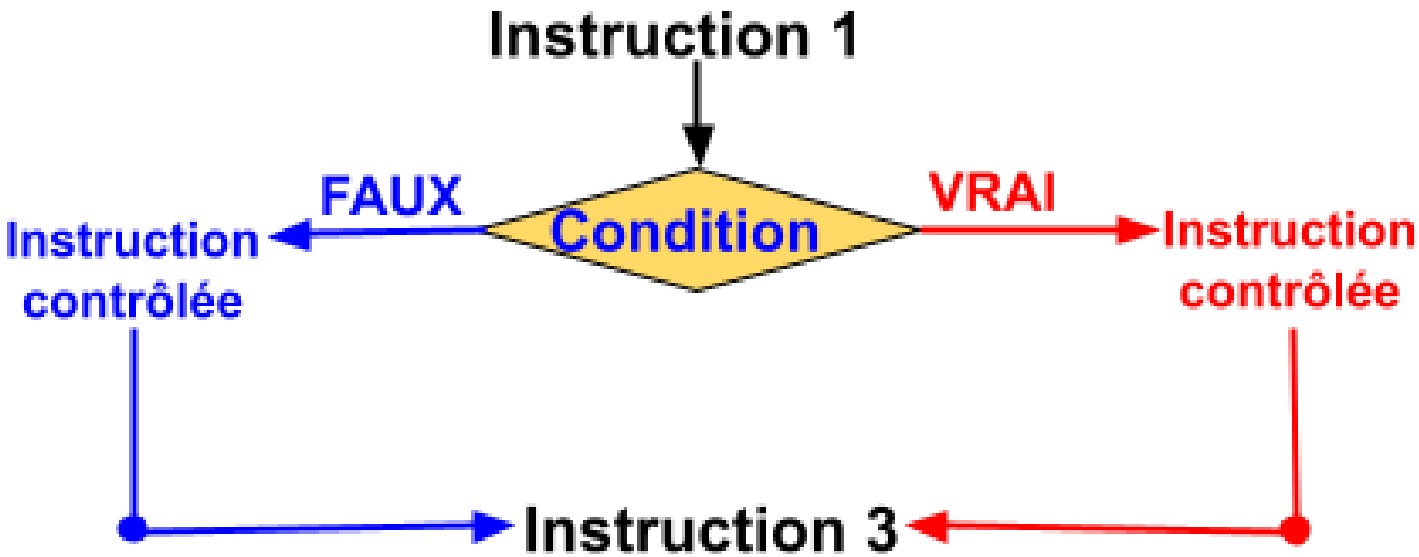


Figure – Organigramme structure alternative

Structure alternative

Syntaxe de la structure alternative

Algorithmique

- DÉBUT
- SI (condition) alors
- instructionVraie
- SINON
- instructionFausse
- FinSI
- FIN

Langage C

- int main( ) {
- if (condition) {
- instructionVraie ;}
- else {
- instructionFausse ; }
- return 0 ;}

Structure alternative

Exemple

Algorithmique

- VAR x :entier
- DÉBUT
- x $\longleftarrow$ 11
- SI (x >= 0) alors
- Écrire(x ,"est positif")
- SINON
- Écrire(x ,"est négatif")
- FinSI
- FIN

Langage C

- short x = 11 ;
- int main( ) {
- if (x >= 0) {
- printf("%hi est positif",x) ;}
- else {
- printf("%hi est négatif",x) ;}
- return 0 ;}

2 Structures de contrôle conditionnelle

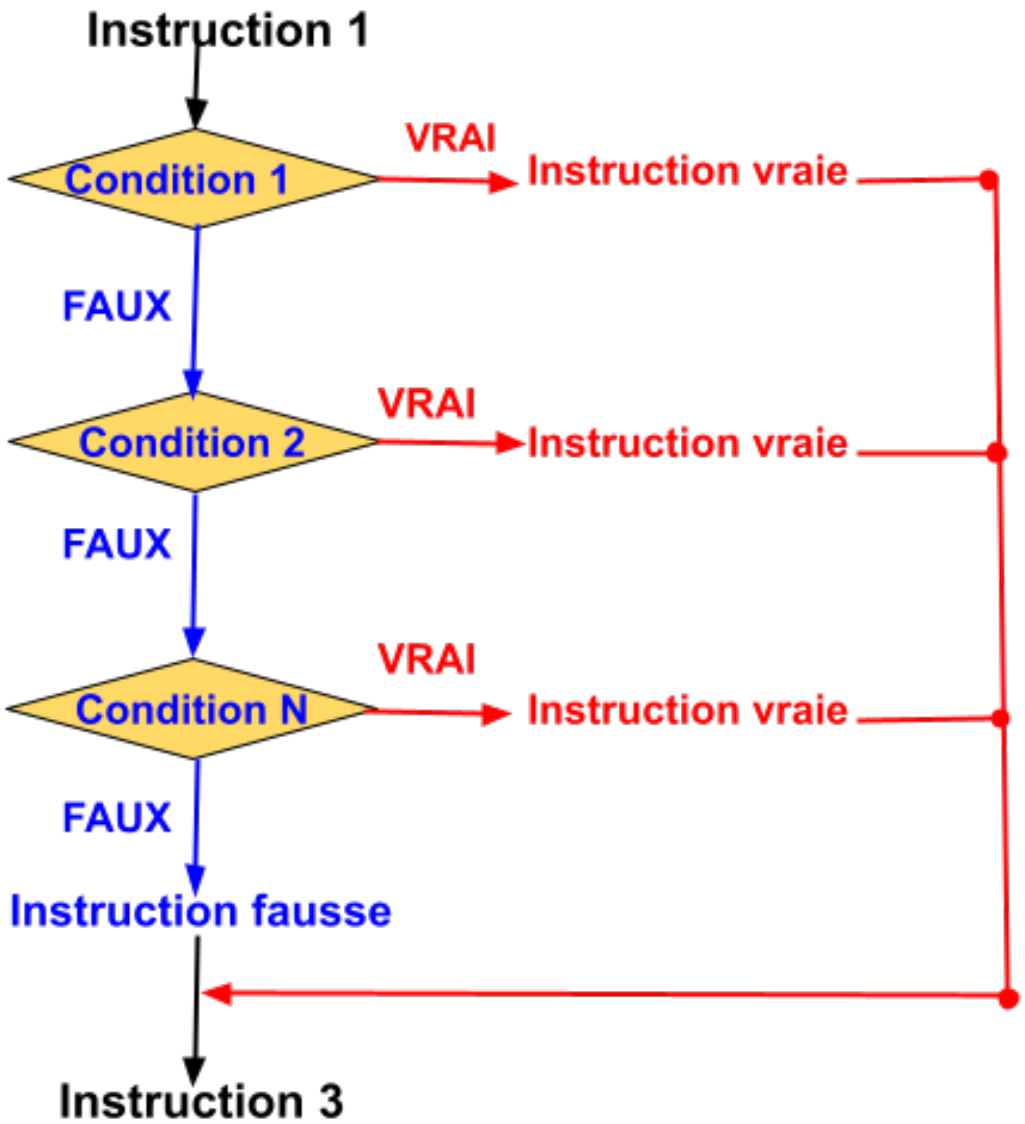
- Définition
- Structure simple
- Structure alternative
- Structure au choix multiple

Structure au choix multiple

Définition

Une structure conditionnelle au choix multiple permet de choisir parmi plusieurs options en fonction de la valeur d'une expression. Chaque condition optionnelle correspond à une valeur possible de l'expression, et l'ensemble d'instructions associé à la valeur correspondante est exécuté.

Structure au choix multiple



Structure au choix multiple

Syntaxe de la structure au choix multiple

SELON (valeur) **Faire**	**switch** (valeur) **{**
cas 1 : instruction 1	**case 1 : instruction 1; break;**
cas 2 : instruction 2	**case 2 : instruction 2; break;**
cas 3 : instruction 3	**case 3 : instruction 3; break;**
SINON	**default : Instruction_fausse;**
Instruction_fausse	
	}
FinSELON	

Structure au choix multiple

Syntaxe de la structure alternative multiple

Algorithmique

- DÉBUT
- SI (condition1) alors
- instructionVraie1
- SINON SI (condition2) alors
- instructionVraie2
- SINON
- instructionFausse
- FinSI
- FIN

Langage C

- int main( ) {
- if (condition1) {
- instructionVraie1 ;}
- else if (condition2) {
- instructionVraie2 ;}
- else {
- instructionFausse ; }
- return 0 ;}

Structure au choix multiple

Algorithmique

- VAR a,b :entier
- c :caractère
- DÉBUT
- a $\longleftarrow$ 9
- b $\longleftarrow$ 4
- c $\longleftarrow$ '-'
- SELON (c) faire
- cas '+' : a $\longleftarrow$ a + b
- cas '-' : a $\longleftarrow$ a - b
- SINON
- Écrire("erreur")
- FinSELON
- FIN

Langage C

- short a = 9, b = 4 ;
- char c ='-' ;
- int main( ) {
- switch (c) {
- case '+' : a=a+b ; break ;
- case '-' : a=a-b ; break ;
- default : printf("erreur") ;}
- return 0 ;}

Structure au choix multiple

- ***Algorithmique***
- VAR x,y,z : entier
- **DÉBUT**
- **SI (x > y et x > z) alors**
- **Écrire("x=",x)**
- **SINON SI (x < y et y > z) alors**
- **Écrire("y = ",y)**
- **SINON**
- **Écrire("z = ",z)**
- **FinSI**

- ***Langage C***
- char x,y,z ;
- **int main( ) {**
- **if (x > y && x > z){**
- **printf("x = %d",x) ;**
- **} else if (x < y && y > z){**
- **printf("y = %d",y) ;**
- **} else {**
- **printf("z = %d",z) ;**
- **}**

3 Structures de contrôle itérative

- Définition
- Structure itérative Pour
- Structure itérative Tant que
- Structure itérative Répéter

Structures itératives

Définition

Certains algorithmes nécessitent de répéter des instructions un certain nombre de fois avant d'obtenir le résultat voulu. Cette répétition est réalisée en utilisant une structure de contrôle itérative nommée boucle.
Il existe principalement trois types de structures itératives :

1. Structure Pour.
2. Structure Tant que.
3. Structure Répéter.

3 Structures de contrôle itérative

- Définition
- Structure itérative Pour
- Structure itérative Tant que
- Structure itérative Répéter

Structure itérative Pour

Définition

La structure de contrôle itérative "Pour" est utilisée pour répéter l'exécution d'un ensemble d'instructions un certain nombre de fois, en spécifiant explicitement la valeur initiale, la condition de continuation et l'opération de mise à jour. Elle est particulièrement utile lorsque vous connaissez à l'avance le nombre d'itérations nécessaire.

Structure itérative Pour

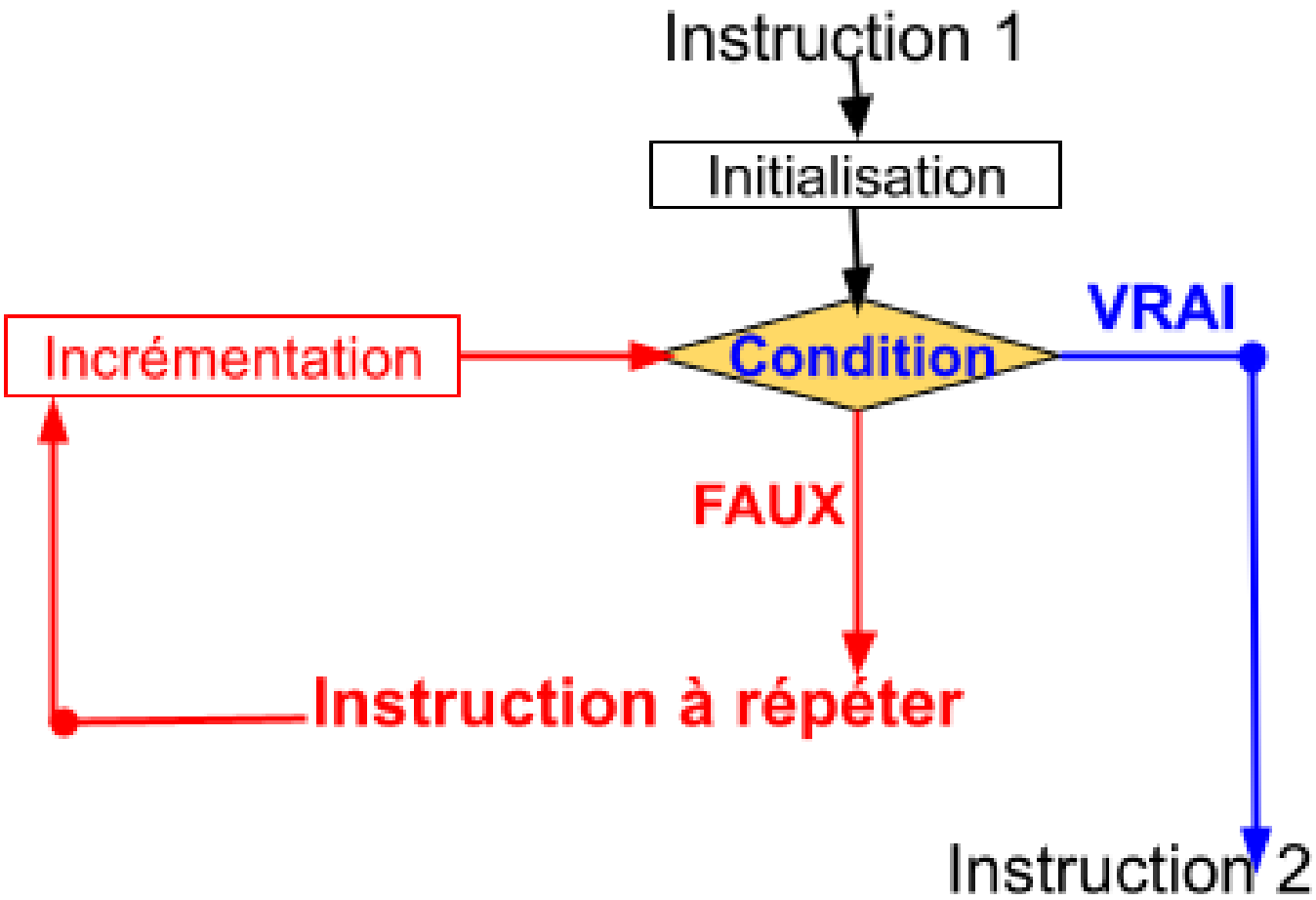


Figure – Organigramme Structure itérative Pour

Structure itérative Pour

Syntaxe de structure itérative Pour

Algorithmique

- DÉBUT
- Pour c de V_i à V_f pas$\longleftarrow$1 faire
- Début
- instructionArépeter
- FinPour
- FIN

Langage C

- int main( ) {
- for (c=V_i ;c<=V_f ;c++)
- { instructionArépeter ;}
- return 0 ;}

Structure itérative Pour

Exemple

Algorithmique

- VAR b,c,f :entier
- DÉBUT
- Lire(b)
- f $\longleftarrow$ 1
- Pour c de 1 à b pas$\longleftarrow$1 faire
- Début
- f $\longleftarrow$ f * c
- FinPour
- FIN

Langage C

- short b,c ;
- long long f = 1 ;
- int main( ) {
- scanf("%*hi*",&b) ;
- for(c=1 ;c<=b ;c++) {
- f = f * c ;}
- return 0 ;}

3 Structures de contrôle itérative

- Définition
- Structure itérative Pour
- Structure itérative Tant que
- Structure itérative Répéter

Structure itérative Tant que

Définition

La structure de contrôle itérative "Tant que" est utilisée lorsque le nombre d'itérations n'est pas connu à l'avance, mais dépend d'une condition initiale. Elle est couramment employée pour parcourir des ensembles de données ou effectuer des tâches répétitives jusqu'à ce qu'une condition initiale soit fausse.

Structure itérative Tant que

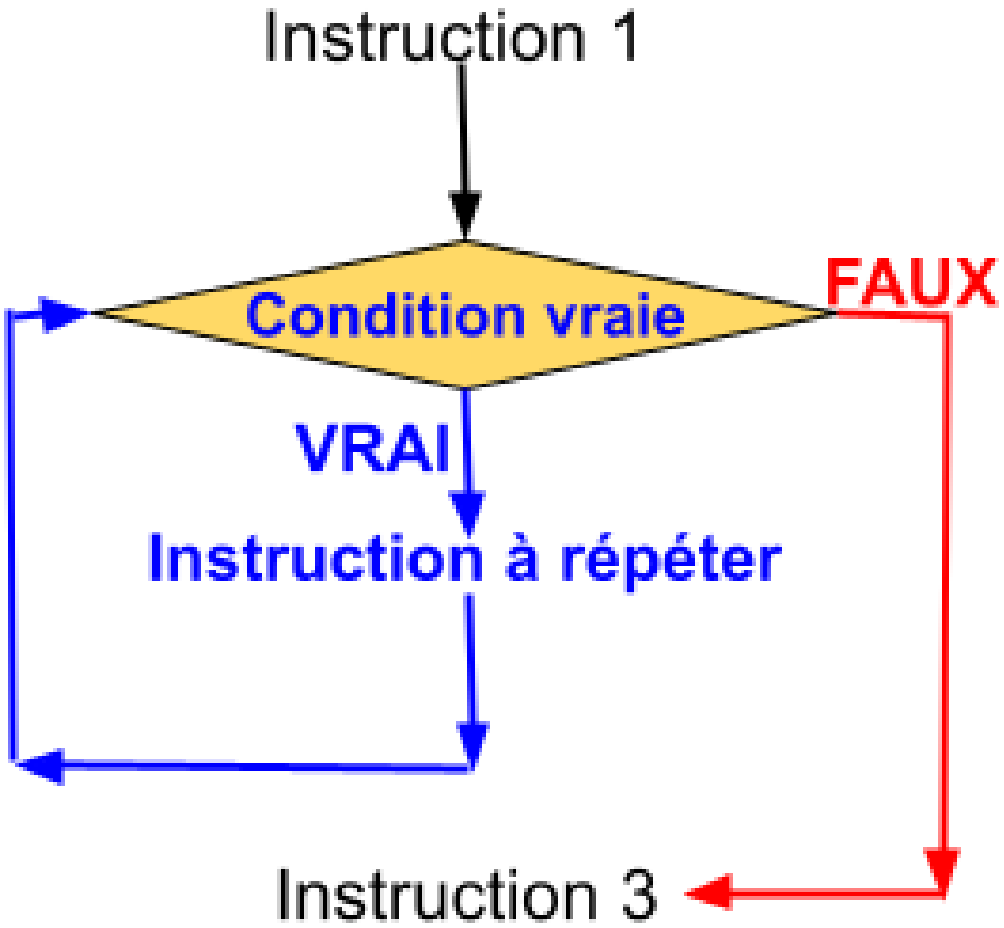


Figure – Organigramme Structure itérative Tant que

Structure itérative Tant que

Syntaxe de structure itérative Tant que

Algorithmique

- DÉBUT
- Tant que (condition)
  faire
- instructionArépeter
- FinTant que
- FIN

Langage C

- int main( ) {
- while (condition) {
- instructionArépeter ;}
- return 0 ;}

Structure itérative Tant que

Exemple

Algorithmique

- VAR x :entier
- DÉBUT
- x $\longleftarrow$ 1
- Tant que (x <= 6) faire
- Écrire("Bonjour")
- x $\longleftarrow$ x + 1
- FinTant que
- FIN

Langage C

- short x = 1 ;
- int main( ) {
- while(x <= 6) {
- printf("Bonjour") ;
- x++ ;}
- return 0 ;}

3 Structures de contrôle itérative
- Définition
- Structure itérative Pour
- Structure itérative Tant que
- Structure itérative Répéter

Structure itérative Répéter

Définition

La structure de contrôle itérative "Répéter jusqu à" est utilisée lorsque vous devez vous assurer que l'ensemble d'instructions est exécuté au moins une fois, même si la condition n'est pas initialement satisfaite. Ensuite, elle continue de répéter tant que la condition contrôlée à la fin reste fausse.

Structure itérative Répéter

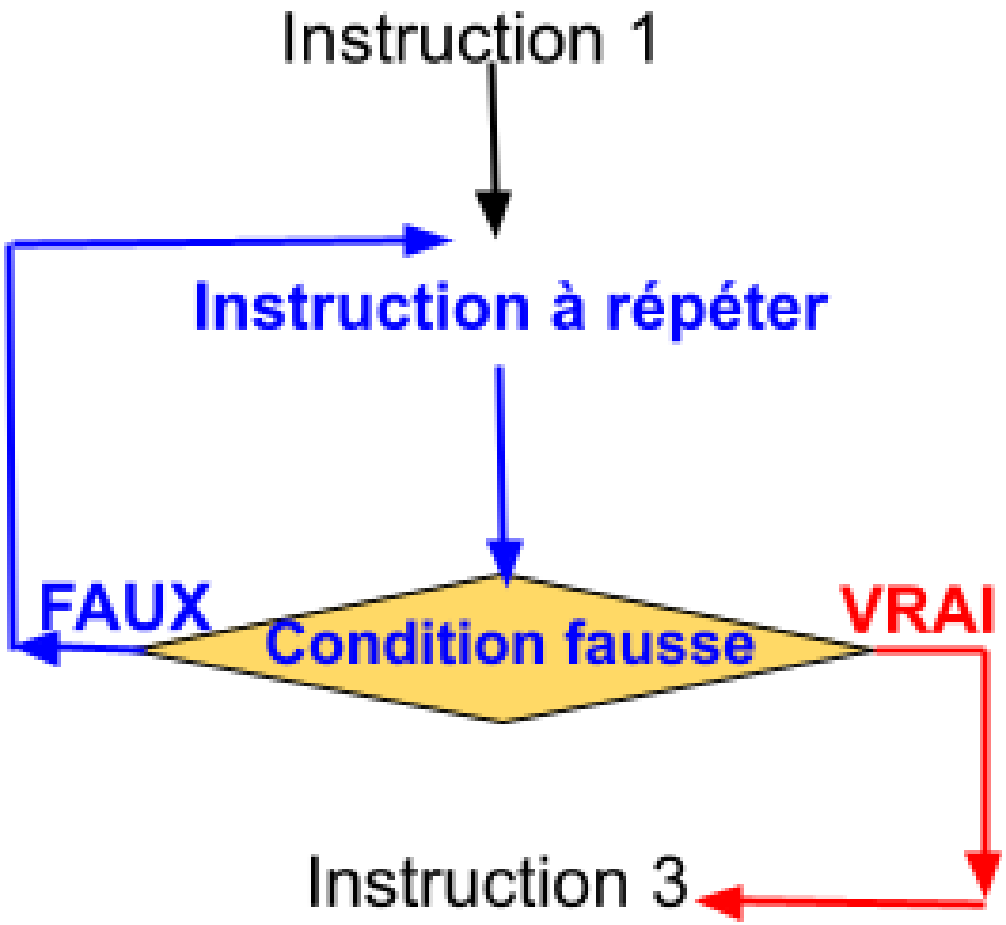


Figure – Organigramme Structure itérative Répéter

Structure itérative Répéter

Syntaxe de structure itérative Répéter

Algorithmique

- DÉBUT
- Répéter
- instructionArépeter
- jusqu'à (condition)
- FIN

Langage C

- int main( ) {
- do {
- instructionArépeter ;
- while (condition) ;}
- return 0 ;}

Structure itérative Répéter

Exemple

Algorithmique

- VAR x :entier
- DÉBUT
- Répéter
- Écrire("Entier positif")
- Lire(x)
- jusqu'à (x >= 0)
- FIN

Langage C

- short x ;
- int main( ) {
- do {
- printf("Entier positif") ;
- scanf("%hi",&x) ;
- }while(x < 0) ;
- return 0 ;}